WEBPACK

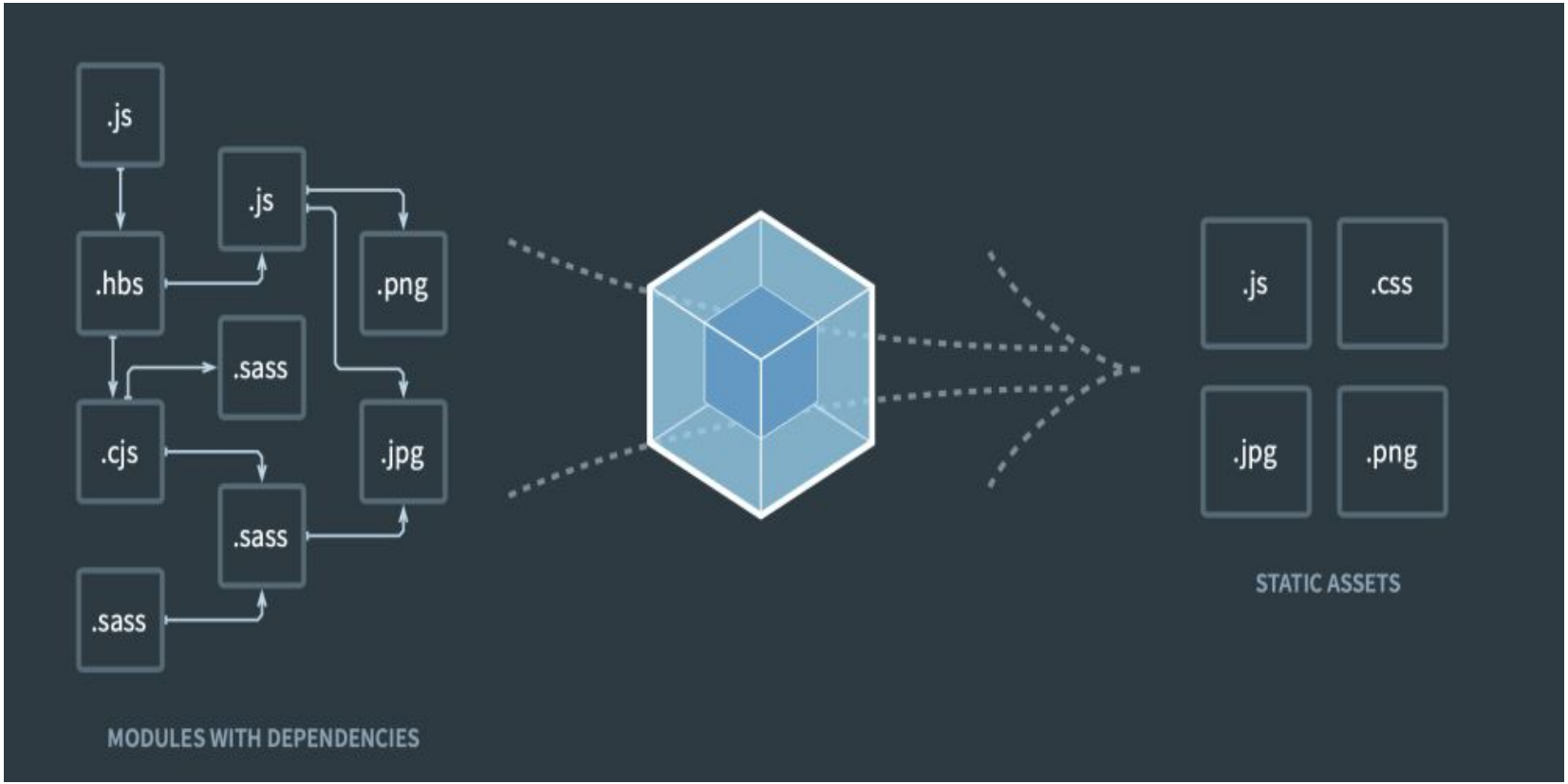
Presented by:
TARUNA SHARMA

What is webpack?

At its core, webpack is a *static module bundler* for modern JavaScript applications.

It takes all the code from your application and makes it usable in a web browser. Modules are reusable chunks of code built from your app's JavaScript, node_modules, images, and the CSS styles which are packaged to be easily used in your website.

Webpack separates the code based on how it is used in your app, and with this modular breakdown of responsibilities, it becomes much easier to manage, debug, verify, and test your code.


.js
.js
.hbs
.png
.sass
.cjs
.jpg
.sass
.sass
MODULES WITH DEPENDENCIES
.js
.css
.jpg
.png
STATIC ASSETS

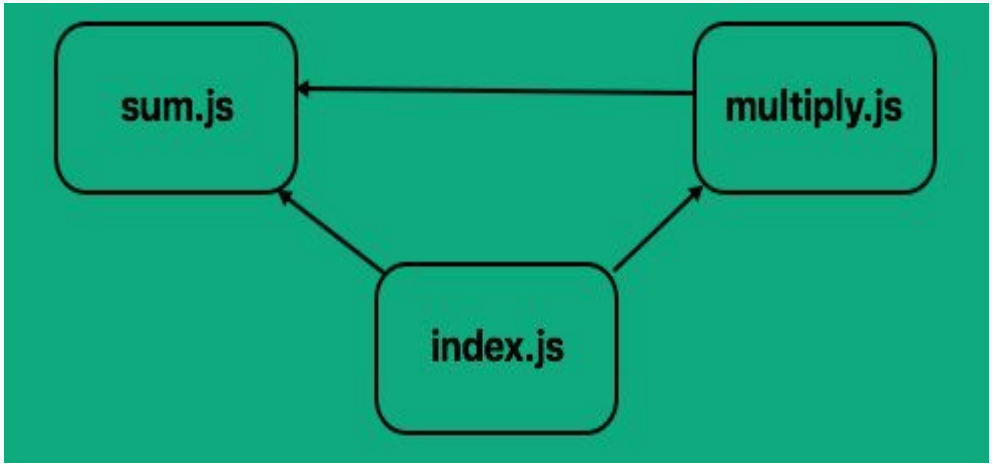
Why do we need webpack?

" **Necessity** is the mother of **invention…"**

Websites before were no more just a small package with an odd number of files in them. They started getting bulky, with the introduction of **JavaScript modules**, as writing encapsulated small chunks of code was the new trend. Eventually all of this lead to a situation where we had 4x or 5x ,more files in our application.
Not only was the overall size of the application a challenge, but also there was a huge gap in the kind of code developers were writing and the kind of code browsers could understand. Developers had to use a lot of helper code called **polyfills** to make sure that the browsers were able to interpret the code in their packages.
To answer these issues, webpack was created.

Dependencies—Modules To the Rescue!

Let's assume we have an application that can perform two simple mathematical tasks—sum and multiply. We decide to split these functions into separate files for easier maintenance.



Main file index.js depends on sum.js and multiply.js.

Multiply.js depends on sum.js.

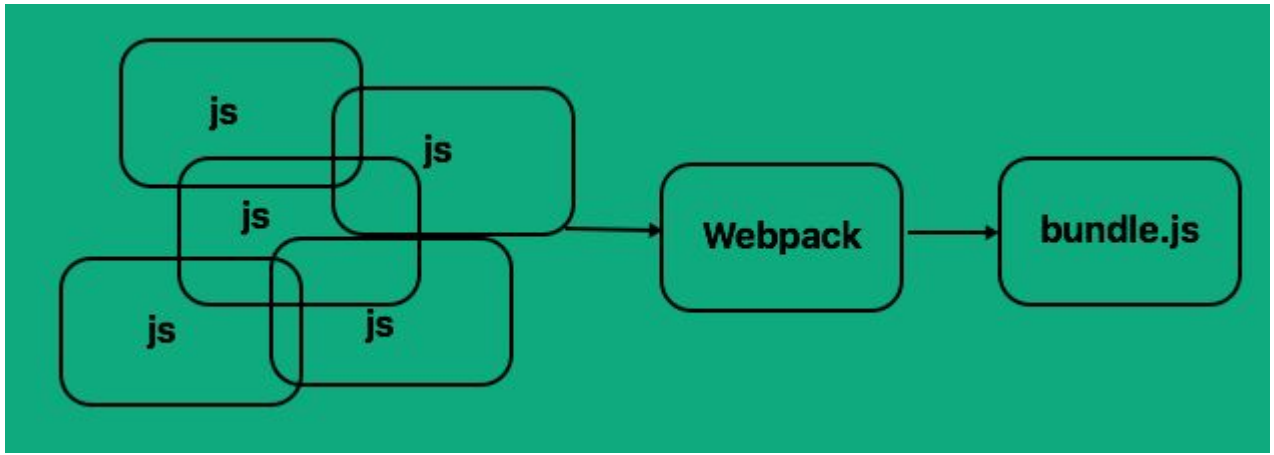
If we get the order wrong in index.html our application won't work. If index.js is included before ***either*** of the other dependencies, or if sum.js is included ***after*** multiply.js we will get errors.

Now imagine that we scale this up to an actual fully blown Web Application — we may have dozens of dependencies, some of which depend on each other. Maintaining order would become a nightmare!

Finally, by using global variables, we risk other code overwriting our variables, causing hard to find bugs.**Webpack can convert these dependencies into modules** — they will have a much tighter scope (which is safer). Additionally by converting our dependencies into Modules, Webpack can manage our dependencies for us — Webpack will pull in the dependant Modules at the right time, in the correct scope.

Death by a Thousand Cuts — Reducing Traffic

If we take a look at index.html, we can see that we'll need to pull down 3 separate files. This is fine but now imagine again that we have many dependencies — the end user would have to wait until all of the dependencies had been downloaded before the main application could run.



The other main feature Webpack offers is bundling. That is, Webpack can pull all of our dependencies into a single file, meaning that only one dependency would need to be downloaded.

Webpack goes through your package and creates what it calls a **dependency graph** which consists of various **modules** which our webapp would require to function as expected.

Two main things that webpack does:

1. It bundles out code together.
2. It manages dependencies -
   It makes sure that code that needs to be loaded first gets loaded first.
   If a file depends on 3 other files than those 3 files need to be included first.

To get started you only need to understand its Core Concepts:

- Entry
- Output
- Loaders
- Plugins
- Mode
- Browser Compatibility

Installing and using webpack

1. Application must contain a package.json file.
2. Than install webpack.

   ~ npm install --save-dev webpack webpack-cli.

3. Create a configuration file: **webpack.config.js**

Webpack's configuration file is a JavaScript file that exports a webpack configuration. This configuration is then processed by webpack based upon its defined properties.

This file basically tells our application where to start from, where to output the bundle and what all loaders and plugins to be used in our application.

Now in order to use the configuration file,we need to mention it in the script tag of our package.json file.

```
"scripts": {
    "start": "webpack --config webpack.config.js"
  }
```

Now when we run npm start the script corresponding to start runs and and the file webpack.config.js mentioned there starts executing.

Entry

An entry point indicates which module webpack should use to begin building out its internal dependency graph.

By default its value is `./src/index.js`, but you can specify a different by setting an `entry` property in the webpack configuration. For example:

```
module.exports = {

  entry: './path/to/my/entry/file.js',

};
```

Output

The output property tells webpack where to emit the *bundles* it creates and how to name these files. It defaults to `./dist/main.js` for the main output file and to the `./dist` folder for any other generated file.

You can configure this part of the process by specifying an `output` field in your configuration:

```
const path = require('path');

module.exports = {
 entry: './path/to/my/entry/file.js',
 output: {
  path: path.resolve(__dirname, 'dist'),        //resolves an absolute path using path module
  filename: 'my-first-webpack.bundle.js',     //the output filename
 },
};
```

In the example above, we use the `output.filename` and the `output.path` properties to tell webpack the name of our bundle and where we want it to be emitted to.

Loaders:

Loaders are transformations that are applied to the source code of a module. They allow you to pre-process files as you `import` or “load” them. Thus, loaders are kind of like “tasks” in other build tools.
Loaders transform your code,so that it can be included inside the javascript bundle.

Loaders can transform files from a different language (like TypeScript) to JavaScript or load inline images as data URLs. Loaders even allow you to do things like `import` CSS files directly from your JavaScript modules!

Example:

For example, you can use loaders to tell webpack to load a CSS file or to convert TypeScript to JavaScript. To do this, you would start by installing the loaders you need:

npm install --save-dev css-loader ts-loader

And then instruct webpack to use the `css-loader` for every `.css` file and the `ts-loader` for all `.ts` files:

```
module.exports = {
 module: {
  rules: [
   { test: /\.css$/, use: 'css-loader' },          // 2   CSS  loader
    { test: /\.ts$/, use: 'ts-loader' },           // 1   Typescript loader
  ],
 },
};
```

Using Loaders

Configuration

`module.rules` allows you to specify several loaders within your webpack configuration. This is a concise way to display loaders, and helps to maintain clean code. It also offers you a full overview of each respective loader.

Loaders are evaluated/executed from right to left (or from bottom to top).

In the example used previously execution starts with typescript loader and than moves on to CSS loader.

Loader features:

- Loaders can be chained. Each loader in the chain applies transformations to the processed resource. A chain is executed in reverse order. The first loader passes its result (resource with applied transformations) to the next one, and so forth. **Finally, webpack expects JavaScript to be returned by the last loader in the chain.**

Some of the useful loaders:

1. **SASS Loader CSS Loader and Style Loader**

   Install the sass loader, the css loader and the style loader. Chain the `sass-loader` with the css-loader and the style-loader to immediately apply all styles to the DOM

```
{
        test: /\.scss$/,            //file ending with .css extension
        use: ['style-loader',       //injects styles into DOM
        'css-loader',               //turns css into common js
        'sass-loader']              //turns sass into css
}
```

2. Html loader

To begin, you'll need to install `html-loader` : ***npm install --save-dev html-loader***

Basically, the HTML loader turns some HTML code:
* Hello <img src="webpack.png" alt="Webpack"> *
into a JavaScript module, like this:
module.exports = " Hello <img src=\"" + require("webpack.png") + "\" alt=\"Webpack\"> ";

```
module.exports = {
 module: {
  rules: [
   {
    test: /\.html$/i,
    loader: 'html-loader',
   },
  ],
 },
};
```

3. File loader

The `file-loader` resolves `import/require()` on a file into a url and emits the file into the output directory.

To begin, you'll need to install `file-loader`:

$ npm install file-loader --save-dev

Then add the loader to your `webpack` config. For example:

```
module.exports = {
 module: {
  rules: [
   {
    test: /\.(png|jpe?g|gif)$/i,
    use: [
     {
      loader: 'file-loader',
     },
    ],
   },
  ],
 },
};
```

Plugins

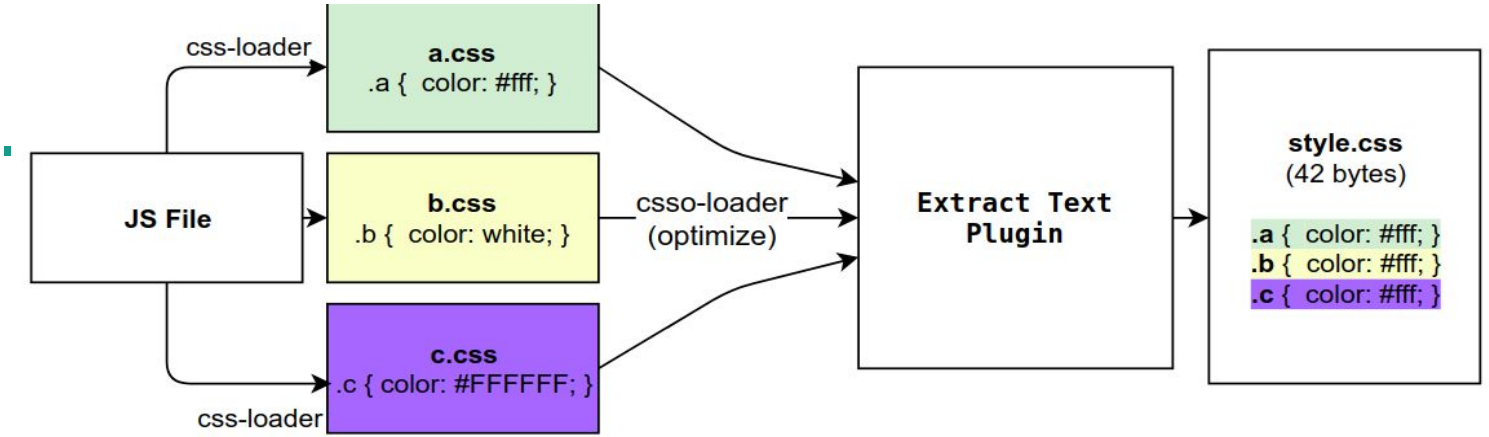
Plugins are the backbone of webpack. Webpack itself is built on the same plugin system that you use in your webpack configuration!

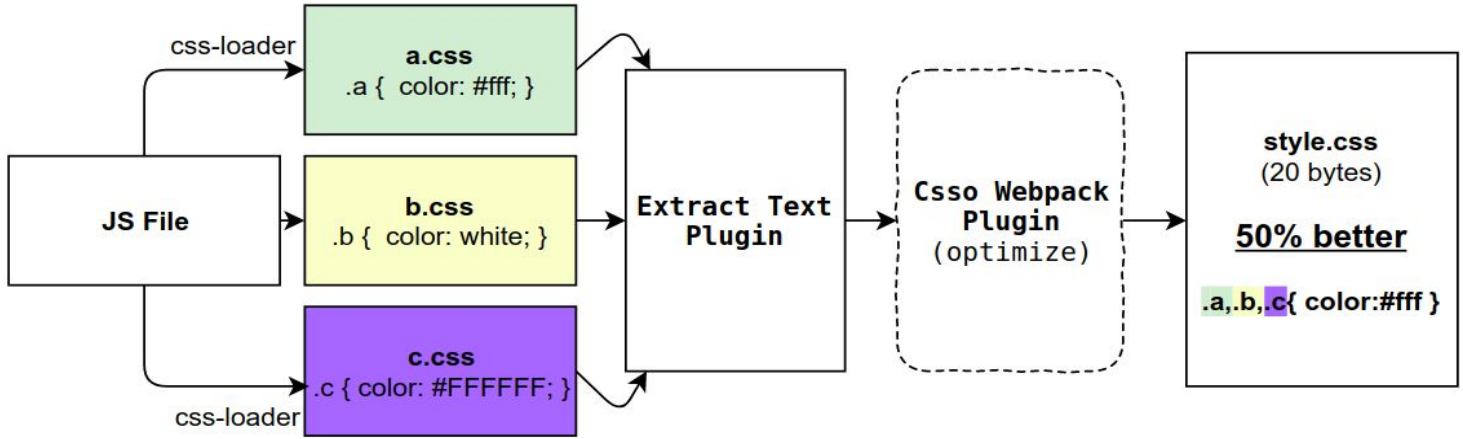
They also serve the purpose of doing anything else that a loader cannot do.

Webpack contains default behaviors to bundle most type of resources. When loaders are not enough, we can use plugins to modify or add capabilities to Webpack.

Plugins work at bundle or chunk level and usually work at the end of the bundle generation process. Plugins can also modify how the bundles themselves are created. Plugins have more powerful control than loaders.

Just for an example you can clearly see in the next image where loaders are working and where plugins are working -


css-loader
a.css
.a { color: #fff; }
JS File
b.css
.b { color: white; }
csso-loader
(optimize)
c.css
.c { color: #FFFFFF; }
css-loader
Extract Text
Plugin
style.css
(42 bytes)
.a { color: #fff; }
.b { color: #fff; }
.c { color: #fff; }


css-loader
a.css
.a { color: #fff; }
JS File
b.css
.b { color: white; }
c.css
.c { color: #FFFFFF; }
css-loader
Extract Text
Plugin
Csso Webpack
Plugin
(optimize)
style.css
(20 bytes)
50% better
.a,.b,.c{ color:#fff }

Generate an `index.html` file

When building single-page applications we usually need one `.html` file to serve it.

The HtmlWebpackPlugin automatically creates an `index.html` file and add script tags for each resulting bundle. It also supports templating syntax and is highly configurable.

```
var HTMLWebpackPlugin = require('html-webpack-plugin');

var baseConfig = {

 // ...

 plugins: [

  new HTMLWebpackPlugin()

 ]

};
```

Providing our own template.html file to the plugin.

```
const HtmlWebpackPlugin = require('html-webpack-plugin');   //installed via npm
const webpack = require('webpack');   //to access built-in plugins
const path = require('path');

module.exports = {
  entry: './path/to/my/entry/file.js',
  output: {
    filename: 'my-first-webpack.bundle.js',
    path: path.resolve(__dirname, 'dist')
  },
    plugins: [
       new HtmlWebpackPlugin({template: './src/index.html'})
  ]
};
```

Some useful plugins:

Define Plugin

The `DefinePlugin` allows you to create global constants which can be configured at compile time.

This can be useful for allowing different behavior between development builds and production builds.

Example:

Providing a global service URL:

```
new webpack.DefinePlugin({

  'SERVICE_URL': JSON.stringify('https://dev.example.com')

});
```

CommonsChunkPlugin

The `CommonsChunkPlugin` is an opt-in feature that creates a separate file (known as a chunk), consisting of common modules shared between multiple entry points.

By separating common modules from bundles, the resulting chunked file can be loaded once initially, and stored in cache for later use. This results in page speed optimizations as the browser can quickly serve the shared code from cache, rather than being forced to load a larger bundle whenever a new page is visited.

```
var CommonsChunkPlugin = require('webpack/lib/optimize/CommonsChunkPlugin');

   plugins: [
       new CommonsChunkPlugin({
            name: 'shared',
            minChunks: 2
       }),
       new HtmlWebpackPlugin({
            template: './src/index.html',
            chunks:'shared',
            filename: './dist/index.html'
       }),
       new HtmlWebpackPlugin({
           template: './src/index.html',
           chunks: 'shared',
           filename: './dist/settings.html'
       })
  ]
```

CopyWebpackPlugin

Copies individual files or entire directories, which already exist, to the build directory.

After installing, add the plugin to your `webpack` config. For example:

Patterns: From: Path from where we copy files, To : Output path.

```
const CopyPlugin = require("copy-webpack-plugin");

module.exports = {

 plugins: [
  new CopyPlugin({
   patterns: [
    { from: "source", to: "dest" },
    { from: "other", to: "public" },
   ],
  }),
 ],
};
```

ExtractText webpack plugin

Extract text from a bundle, or bundles, into a separate file.

```
module.exports = {
  module: {
    rules: [
      {
        test: /\.css$/,
        use: ExtractTextPlugin.extract({                    (1)
          fallback: "style-loader",
          use: "css-loader"
        })
      }
    ]
  },
  plugins: [
    new ExtractTextPlugin("styles.css"),                    (2)
  ]
}
```

At line 1: This tells Webpack to pass the results off the css-loader to the ExtractTextPlugin. At the bottom we configure the plugin.

At line 2: this does is tell the plugin that for all data passed to it, save it down to a file called style.css.

It moves all the required `*.css` modules in entry chunks into a separate CSS file. So your styles are no longer inlined into the JS bundle, but in a separate CSS file (`styles.css`). If your total stylesheet volume is big, it will be faster because the CSS bundle is loaded in parallel to the JS bundle.

Mode:

Webpack has two modes of operations: **development** and **production**.

In **development** mode, webpack takes all the JavaScript code we write, almost pristine, and loads it in the browser.

No **minification** is applied. This makes reloading the application in development faster.

In **production** mode instead, webpack applies a number of optimizations:

- minification with TerserWebpackPlugin to reduce the bundle size
- scope hoisting with ModuleConcatenationPlugin

It also set `process.env.NODE_ENV` to "production". This environment variable is useful for doing things conditionally in production or in development.

```
"scripts": {

  "dev": "webpack --mode development",        //npm run dev     (Development mode)

  "build": "webpack --mode production"        //npm run build   (Production mode)

 }
```

Modules

In modular programming, developers break programs up into discrete chunks of functionality called a *module*.

What is a webpack Module

In contrast to Node.js modules, webpack *modules* can express their *dependencies* in a variety of ways. A few examples are:

- An ES2015 `import` statement
- A CommonJS `require()` statement
- An AMD `define` and `require` statement
- An `@import` statement inside of a css/sass/less file.
- An image url in a stylesheet `url(...)` or HTML `<img src=...>` file.

Asset management

Most often your project will contain assets such as images, fonts, and so on. In webpack 4, to work with assets, we had to install one or more of the following loaders: `file-loader`, `raw-loader`, and `url-loader`. In webpack 5, this is not needed anymore, because the new version comes with the built-in asset modules.

Here, we'll explore an example with images. Let's add new rule in the `webpack.config.js`:

```
module: {
 rules: [
   ...
  {
    test: /\.(?:ico|gif|png|jpg|jpeg)$/i,
    type: 'asset/resource',
  },
 ]
}
```

Here, the type `asset/resource` is used instead of `file-loader`.

How webpack works with angular?

What is Angular CLI ?

Angular CLI is a command-line interface (CLI) to automate your development workflow. It allows you to:

- create a new Angular application
- run a development server with LiveReload support to preview your application during development
- add features to your existing Angular application
- run your application’s unit tests
- run your application’s end-to-end (E2E) tests
- build your application for deployment to production.

Creating project with Angular CLI

1. Install Angular CLI: `npm install -g @angular/cli`

   This will install the ng command globally on our system.

2. Create a new Angular application

   ```
   ng new my-app
   ```

   Behind the scenes, the following happens:

- a new directory my-app is created
- all source files and directories for your new Angular application are created based on the name you specified (my-app)
- npm dependencies are installed
- TypeScript is configured for you
- the Karma unit test runner is configured for you
- the Protractor end-to-end test framework is configured for you
- environment files with default settings are created.

3. Run your application:

```
$ ng serve
```

Behind the scenes, the following happens:

1. Angular CLI loads its configuration from .angular-cli.json
2. Angular CLI runs Webpack to build and bundle all JavaScript and CSS code
3. Angular CLI starts Webpack dev server to preview the result on localhost:4200.

As we read that to bundle and configure all the files,it is mandatory to have webpack.config.js file.

But wait,I just searched my entire Angular project and there is no webpack.config.js. Are you sure Angular CLI still uses webpack?

OK, you're right, there is no webpack.config.js in our Angular project.

But, let's take a quick look and verify that Angular does in fact depend on webpack.

In our project we can find a local copy of the Angular CLI here:

`node_modules\@angular\cli`

In that directory you should see the package.json file for Angular CLI.

Open it and take a look at the dependencies and among others you will see webpack, webpack-dev-server, etc.

Also, in node_modules you can see that these webpack based dependencies actually did get installed.

So, we see that Angular CLI is, in fact, using webpack but it is only a black box to us.

The Angular CLI hides all that webpack complexity. However, that simplicity comes at the price of flexibility. By using the Angular CLI you lose the ability to configure and customize webpack.

References

https://medium.com/a-beginners-guide-for-webpack-2/common-chunks-ba2b4335caea

https://www.npmjs.com/package/extract-text-webpack-plugin

https://blog.ag-grid.com/webpack-tutorial-understanding-how-it-works/

https://survivejs.com/webpack/what-is-webpack/#:~:text=Webpack%20gives%20you%20control%20over,to%20couple%20styling%20with%20components.

https://www.twilio.com/blog/2018/03/building-an-app-from-scratch-with-angular-and-webpack.html